BlockTrack

BLOCKCHAIN-BASED PARCEL TRACKING
AND DELIVERY MANAGEMENT DAPP

ACADEMIC PROJECT · 2026

BlockTrack

Parcel tracking you can verify. Details only the right people can read.

A privacy-preserving parcel tracking and delivery management DApp. The chain holds a tamper-evident audit trail; the browser holds the keys. No backend and no plaintext shipment records.

34	21	2/2	0
contract tests passing	end-to-end browser checks	Caliper rounds passed	plaintext records on-chain

PROJECT REPORT · 2026

PROJECT TEAM — REGISTRATION NO.

Shantanu Bhargava 230957130	Ankur Kumar 230957186	Rijul Kaushik 230957070
Ananya Sahani 230957046	Hemesh Kukreja 230957048	Sandeep Balaji 230957140
Adhisha Sahoo 230957024	Kshitij Luthra 230968098	Vansh Pahwa 230968346

Foundry · React · Ethereum · Hyperledger Caliper

October 2026

BLOCKTRACK · PROJECT REPORT

Contents

A privacy-preserving parcel tracking DApp. Final system, design decisions, evidence and team contributions.

01 Executive summary 2

02 Introduction and problem statement 3

03 Scope and objectives 3

04 System architecture 5

05 Smart contract design 6

06 Privacy and encryption design 9

07 Application and user experience 11

08 Implementation and technology choices 16

09 Key decisions and trade-offs 17

10 Testing and quality assurance 19

11 Benchmarking with Caliper 20

12 Deployment and limitations 21

13 Team contributions 23

14 Conclusion 23

15 References 24

About this report. This document describes the final system that was built: its architecture, contract, privacy model, application, implementation, testing, benchmark, deployment and team contributions. It is written for academic review.

01 Executive summary

BlockTrack is a working Ethereum DApp for parcel tracking that treats privacy as part of the design rather than an afterthought. The chain keeps a tamper-evident record of who changed a shipment and when; shipment details, addresses and courier notes remain encrypted in the browser.

The application has four experiences: a customer who creates and tracks orders, an operations admin who dispatches and manages the courier team, a courier who updates only assigned shipments, and a public tracker that reveals status and timestamps but no personal data. There is no backend, database or account server. The frontend is a static React application that talks directly to an Ethereum JSON-RPC node.

THE DESIGN THESIS

Keep the audit trail public, keep the payload private, and let encryption — not a login screen — decide who can read an order.

34/34	7/7	21/21	2/2
Foundry contract tests	Cryptographic unit tests	Browser end-to-end checks	Caliper rounds, no failures

What was delivered

- A Solidity contract with role-based access, customer/admin/courier order flows, a seven-state status machine, encrypted payloads and per-order key envelopes.
- A browser-side cryptographic layer: AES-256-GCM per order, ECIES key wrapping, and a vault key derived from a wallet signature through HKDF-SHA256.
- A complete React product with 15 page components across 17 route entries, a custom Cold Precision design system, light/dark themes, responsive navigation and mobile layouts.
- Deterministic demonstration data: 10 seeded orders, 7 courier assignments and 21 timeline updates across six fictional local accounts.
- Verification evidence: 34 contract tests, 7 crypto tests, a 21-check browser flow, 24 captured UI views and a two-round Caliper benchmark.
- Documentation covering the privacy model, architecture, testing, benchmark and the final report.

Result

The delivered system supports the full shipment lifecycle: a customer creates an encrypted order, operations assigns a courier and wraps the order key, the courier updates the status and adds an encrypted note, the customer reads the update, and the public verifies status without seeing personal data. Every status change is a contract transaction; every read is decrypted locally.

SUMMARY

The project delivers a coherent privacy-first prototype, a complete role-aware interface, reproducible tests and measured contract evidence. It is a proof of concept, not a production logistics platform.

02 Introduction and problem statement

Parcel tracking looks simple from the outside: a tracking number, a status, a timeline. The trust model behind it is not simple. The same organisation that records a delivery also controls the database, the permissions and the history.

The original problem

Centralised tracking systems can be changed by the organisation that operates them. An administrator can correct, backdate or remove a status update. For most day-to-day logistics this is acceptable; for disputes, audits and multi-party supply chains it is a weakness. A shared, append-only record makes the history independently verifiable and gives every participant the same view of what happened.

The problem the implementation uncovered

A naive blockchain solution fixes integrity but breaks privacy. If parcel records, names, phone numbers and addresses are written to a public ledger, the audit trail becomes a permanent public data leak. The project therefore had to answer a sharper question:

RESEARCH QUESTION

Can a public blockchain provide a tamper-evident parcel audit trail while keeping shipment details readable only by the customer, operations and the assigned courier?

BlockTrack answers with an encrypted envelope model. The chain stores ciphertext, status, timestamps and wrapped key envelopes. Authorised readers unwrap the per-order key in their browser. The public tracker uses separate getters that return status and timestamps only.

Contributions

- **A protocol design** that separates write authorisation (on-chain roles) from read authorisation (cryptographic key envelopes).
- **A complete role-aware product** for customers, operations and couriers, plus a deliberately limited public view.
- **A client-side key hierarchy** built on wallet signatures, HKDF, ECIES and AES-256-GCM, with no server or password.
- **An evidence base** of contract, crypto, browser and benchmark tests that is reproducible on a single machine.
- **An honest record of trade-offs** — including the limits the prototype does not solve (public metadata, post-compromise revocation and the trusted admin key).

03 Scope and objectives

The project scope is the implemented system: a privacy-preserving parcel tracking DApp with customer, operations, courier and public experiences, backed by a single Solidity contract and a static React frontend.

3.1 Scope

In scope

- ✓ Customer, admin/operations and courier accounts with on-chain roles.
- ✓ Customer order creation and admin-created orders for customers.
- ✓ Seven-state shipment lifecycle with enforced transitions.
- ✓ Encrypted payloads, encrypted notes, wrapped key envelopes and local decryption.
- ✓ Dispatch, courier assignment and reassignment.
- ✓ Authenticated timeline and public status-only tracking.
- ✓ Local Anvil deployment with seeded demonstration data.
- ✓ Contract, crypto and browser tests; Caliper microbenchmarks; documentation.

Out of scope

- Real courier integration, live GPS or route optimisation.
- Payments, escrow, tokens, NFTs or supply-chain automation.
- A production backend, database, email/SMS notifications or server search.
- A public-testnet deployment; the submitted build runs on a local Anvil chain.
- Regulatory compliance, audits or real personal data.

3.2 Objectives and outcomes

Objective	Final outcome	Status / evidence
Build an Ethereum DApp	React SPA plus Solidity contract running against Anvil; full customer, admin, courier and public experiences.	`Achieved` 17 route entries; production build passes.
Record status updates on-chain	Every transition is a transaction; the contract stores status, actor and timestamp and emits events.	`Achieved` 21 seeded updates; 7-state machine.
Verifiable history	Authenticated timeline with actor names, timestamps and encrypted notes; public status/timestamps.	`Achieved` timeline and public getters.
Restrict updates to authorised accounts	Owner/admin for cancellation; assigned courier or admin for updates; OpenZeppelin roles for courier management.	`Achieved` contract tests and end-to-end flow.
Track by unique ID	Public tracker and customer order list use the tracking ID; IDs are unique per order.	`Achieved` public page and browser checks.
Protect shipment privacy	Per-order AES-256-GCM, ECIES key envelopes, signature-derived vault, status-only public view.	`Achieved` 7 crypto tests and privacy page.
Deliver a complete user flow	Customer creation, admin dispatch/team/reports, courier updates, notifications, settings and public tracking.	`Achieved` 21 end-to-end checks.
Test the system	34 contract tests, 7 crypto tests, 21 browser checks, desktop/dark/mobile visual QA.	`Achieved` testing section.
Benchmark the contract	Hyperledger Caliper 0.6.0: 50 order writes and 100 public reads, no failures.	`Achieved` benchmarking section.

Objective	Final outcome	Status / evidence
Document the system	README, privacy model, benchmark page and this report.	`Achieved` project documentation.

The objectives are met by the submitted implementation. The system is a proof of concept; it does not connect to real couriers or use real personal data.

3.3 The delivered system

The final system is a single-page React application served from static files, a Solidity contract deployed to a local Anvil chain, and a browser-side cryptography layer. It demonstrates the complete order lifecycle, role enforcement, encrypted access and a public status view. The contract and frontend are chain-agnostic: the network, RPC endpoint and contract address are runtime configuration, not application logic.

04 System architecture

BlockTrack is a static browser application and a Solidity contract. There is no application server between them. The browser owns the wallet, the vault and the crypto; the chain owns the shared audit trail and the role checks.

FIGURE 1 · SYSTEM ARCHITECTURE



No backend, API, database, cookie session or password store. The browser is the only place where a plaintext order exists.

4.1 Components

Layer	Responsibility
Wallet context	Connects a local account or an injected wallet; tracks chain ID, network mismatch and signer; owns no encryption keys.
Vault context	Signs the fixed vault message, derives the vault key through HKDF, registers/compares the public key, and clears decrypted state on lock or account change.
Crypto module	Pure functions for AES-GCM payload/note encryption, ECIES wrapping, order-key generation and AAD binding.
Data hooks	TanStack Query hooks over ethers: profiles, roles, orders, histories and notifications. No server cache; the query cache is the local cache.
Contract	Roles, profiles, orders, status transitions, history, key envelopes and public getters. It never decrypts.

Layer	Responsibility
Chain	Shared, append-only state and events. Anvil for the local deployment; any EVM RPC in principle.

4.2 Request and data flow

1. **Boot.** The app loads `deployment.json`, connects the RPC, restores a local account or an injected wallet, and waits for a vault unlock.
2. **Unlock.** The wallet signs the fixed message; the browser derives the vault key; the public half is compared with the on-chain profile.
3. **Create.** The browser generates a tracking ID and an order key, encrypts the payload, wraps the key for the owner and admins, sends one transaction.
4. **Dispatch.** The admin unwraps the order key, wraps it for the courier, and calls `assignCourier`; only the new envelope is stored.
5. **Update.** The courier or admin encrypts a note and calls `updateStatus`; the contract enforces the transition and appends history.
6. **Read.** The browser reads events and orders, unwraps its own key, decrypts locally, and derives dashboards, timelines and notifications.
7. **Public.** The tracker calls status-only getters; no ciphertext, key envelope or PII is requested.

4.3 Design principles

Chain as the shared truth

Addresses, status, timestamps and key envelopes are the source of truth. The UI is a derived view, not a second database.

Confidentiality is cryptographic

The contract cannot enforce read access. Read access is enforced by who holds a wrapped order key; the public getters are a separate, deliberately limited surface.

Writes are on-chain permissions

Roles and state transitions are contract rules. A forged UI request cannot bypass them.

The local chain is deterministic

Anvil, fixed personas and a seeded ten-order timeline make every screenshot and test reproducible; the production path stays chain-agnostic.

4.4 Why there is no backend

A backend would be the natural place for indexing, notifications and account management, but it would also be the natural place for plaintext. The project deliberately removes that trust surface. The cost is that the browser filters events, computes aggregates locally and stores read state in local storage. At ten orders that is instant; at production scale it would need an indexer that reads only non-sensitive fields.

05 Smart contract design

The contract is intentionally small in scope and strict about state. It stores the order skeleton, encrypted blobs and key envelopes; it does not store names, addresses, parcel text or notes in plaintext.

5.1 Roles and permissions

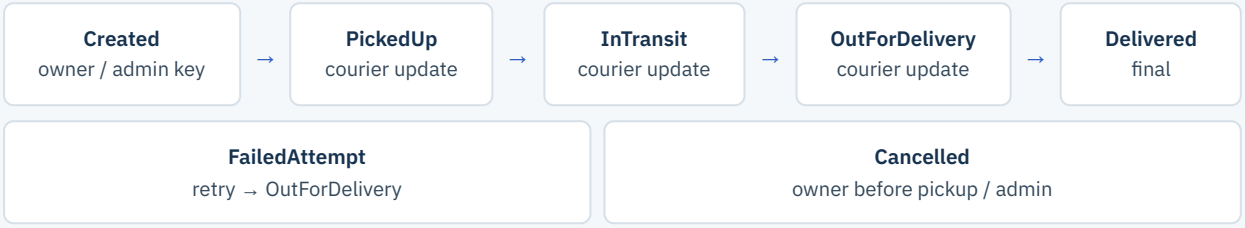
Role	Can do	Cannot do
Admin OpenZeppelin DEFAULT_ADMIN_ROLE	Add/remove couriers; create for a customer; assign/reassign; update any non-final order; cancel any non-final order; read every order key.	Bypass the status state machine; decrypt data for which no envelope exists.
Courier LOGISTICS_ROLE	Read assigned orders; update status on assigned orders; add encrypted notes.	Update an unassigned order; cancel; add/remove roles; read unassigned payloads.
Customer owner address	Create orders; read own orders; cancel before pickup.	Update status; read another customer's order.
Public	Read tracking status and timestamps by ID.	Read payload, notes or actor identity through the public interface.

5.2 Data model

Structure	Fields	Purpose
Profile	`displayName`, `encryptionKey`, `exists`	Public display name plus the 65-byte public key used to wrap order keys.
Order	`trackingId`, `owner`, `courier`, `status`, `createdAt`, `updatedAt`, `ciphertext`, `exists`	The public skeleton plus the encrypted payload.
StatusUpdate	`status`, `timestamp`, `actor`, `encryptedNote`	Append-only history; notes are encrypted.
Wrapped keys	`orderKey → grantee → bytes`; `orderKey → address[]`	One ECIES envelope per authorised reader; a list for discovery and revocation.

5.3 Status machine

FIGURE 2 · SEVEN-STATE SHIPMENT LIFECYCLE



From OutForDelivery the shipment can also move to **FailedAttempt**, and from FailedAttempt back to OutForDelivery for a retry. **Cancelled** is reachable by the owner only while Created, and by an admin on any non-final state. Delivered and Cancelled are final.

5.4 Main functions

Function	Caller	Behaviour
`registerProfile`	any wallet	Stores a display name and a 65-byte public key once; rejects duplicate profiles and invalid keys.

Function	Caller	Behaviour
`updateDisplayName`	profile owner	Changes only the public display name.
`createOrder`	customer	Creates an order owned by the caller. The holder list must include the owner and the primary admin.
`createOrderFor`	admin	Creates an order for a customer, with the same holder requirements.
`assignCourier`	admin	Checks the courier role, stores the new wrapped key, removes the previous courier's envelope.
`updateStatus`	assigned courier / admin	Enforces the state machine, records the actor and timestamp, stores the encrypted note.
`cancelOrder`	owner (Created only) / admin	Final cancellation with an encrypted reason; owner cannot cancel after pickup.
`getPublicStatus`, `getPublicHistory`	anyone	Return status and timestamps only — the public tracker's entire data surface.
`getWrappedKey`, `getKeyHolders`, `hasAccess`	anyone	Return opaque envelopes and holder addresses; access is proven by whether the caller can unwrap.

5.5 Limits and events

Enforced limits

- Tracking ID: 1–32 bytes.
- Ciphertext: up to 8 KiB.
- Encrypted note: up to 2 KiB.
- Wrapped key: 1–256 bytes.
- Display name: 1–48 bytes.
- Public key: exactly 65 bytes, starting with `0x04`.

Events for discovery

`OrderCreated`, `CourierAssigned`, `StatusUpdated`, `KeyGranted`, `KeyRevoked`, `ProfileRegistered`, `DisplayNameUpdated`, `LogisticsAdded`, `LogisticsRemoved`.
The frontend queries events rather than maintaining a server index.

5.6 Security and engineering notes

- **Holder validation.** The contract requires the owner and the primary admin in the holder list, rejects duplicates and zero addresses, and validates every envelope length. This stops an order from being created without an operations recovery key.
- **State finality.** Delivered and Cancelled are final; every update checks the legal transition before writing. The UI mirrors the same rules but the contract is authoritative.
- **No funds, no external calls.** The contract is not payable and makes no external calls, removing the usual reentrancy and value-transfer attack surface.
- **Custom errors.** A compact error surface (for example `MissingOwnerKey`, `InvalidStatusTransition`) makes failed transactions easier to diagnose and cheaper than string reverts.
- **Bytecode margin.** The runtime is **23,657 bytes** against the 24,576-byte EIP-170 limit; the creation bytecode is 24,348 bytes. The margin is under 1 KB, so further features should first reduce or externalise code.
- **Confidentiality is not verified on-chain.** The contract validates lengths, not encryption. Integrity is enforced by AES-GCM authentication and the tracking-ID AAD in the client; a malformed ciphertext is rejected at decryption.

06 Privacy and encryption design

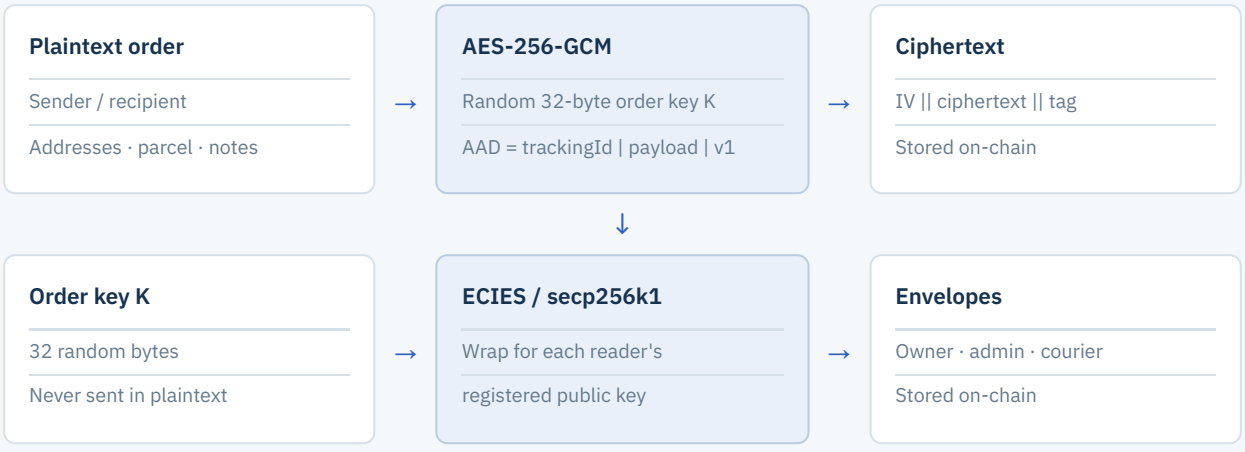
The privacy model is the intellectual core of the project. It separates *who can write* (on-chain roles) from *who can read* (cryptographic key envelopes), and keeps the public ledger to status and timestamps.

6.1 Threat model

Actor	Can observe / do	Cannot do
Passive chain observer	Addresses, status, timestamps, ciphertext lengths and public profiles.	Read names, addresses, parcel text or notes.
RPC provider	Addresses, timing, ciphertext and the browser's IP.	Decrypt payloads.
Another customer	Their own order payloads; public status of any order.	Unwrap someone else's order key.
Unassigned courier	Their own profile and any public status.	Read the payload of an unassigned order; update it.
Operations admin	Every order payload and note by design.	Alter history or bypass the contract state machine.
Compromised browser	Whatever the tab can access while unlocked.	Nothing; the app cannot protect a compromised device.

6.2 Key hierarchy

FIGURE 3 · THE ENVELOPE MODEL



6.3 Vault key derivation

FIGURE 4 · FROM A WALLET SIGNATURE TO AN ENCRYPTION KEY



6.4 Cryptographic choices

Primitive	Use	Why
AES-256-GCM	Payload and note encryption; fresh 12-byte IV per envelope; 16-byte tag.	Authenticated encryption in WebCrypto; detects tampering and wrong AAD.
HKDF-SHA256	Stretch the 65-byte signature into a 32-byte secp256k1 key.	Domain-separated, deterministic, standard KDF; avoids using a wallet signature directly as a key.
secp256k1 / ECIES	Wrap the order key for each reader; unwrap with the vault key.	Ethereum-native curve; one key pair per user; no server key store.
AES-GCM AAD	`${trackingId}\|payload\|v1` and `${trackingId}|note|v1`.	Binds ciphertext to one order and one purpose; prevents moving a valid envelope to another ID.
EIP-191	The fixed vault message signature.	Standard `personal_sign` flow supported by MetaMask and ethers.

6.5 What is public and what is encrypted

Public on-chain

- Tracking ID and order existence.
- Owner and courier addresses.
- Status transitions and timestamps.
- Display names and encryption public keys.

Encrypted / private

- Sender and recipient names.
- Phone numbers and full addresses.
- Parcel description, category, weight, value.
- Delivery instructions.

- Ciphertext length and key-envelope count.
- Caliper benchmark contract (separate scratch deployment).

- Courier notes, locations, ETAs and proof of delivery.
- Cancellation reason.

6.6 Who can read what

Reader	How they get the key	Scope
Customer	Envelope wrapped at creation for their public key.	Own orders only.
Admin	Envelope wrapped at creation for the primary admin; required as a holder.	Every order, for dispatch and support.
Courier	Envelope wrapped by the admin at assignment.	Only assigned orders; loses the envelope on reassignment.
Public	No envelope exists.	Status and timestamps only.

6.7 Revocation, recovery and metadata

- **Reassignment** removes the previous courier's on-chain envelope but cannot erase plaintext already read. True revocation needs a new order key, re-encryption and re-wrapping — documented as future work.
- **Lost wallet, lost key.** There is no recovery server. Operations can still recover the shipment operationally because it holds a key, but that is governance, not cryptographic user recovery.
- **Metadata remains public.** Addresses, timing, order frequency and ciphertext size can be correlated. Encryption hides content, not the existence of a relationship.
- **Admin trust is explicit.** Operations can read every order by design. This is the largest trust boundary and the most important caveat to state.
- **No key rotation.** The vault key is stable for a wallet. Rotating it requires re-encrypting or re-wrapping every order.

> HONEST CLAIM
>
> BlockTrack protects shipment content from other customers, unassigned couriers and the public ledger. It does not hide the fact that an address is involved in a shipment, and it does not remove the need to trust operations.

07 Application and user experience

The DApp is not a single admin screen with a tracking box. It is a complete role-aware product: customers create and follow shipments, operations runs the network, couriers move parcels, and the public verifies status.

7.1 Four experiences

Role	Primary job	Key surfaces
Customer	Create an order, follow its progress, read courier notes, cancel before pickup.	Dashboard · My orders · New order · Order detail · Notifications · Settings
Admin	Dispatch orders, manage couriers, update any non-final shipment, review aggregates.	Dashboard · All orders · Dispatch · Team · Reports · Settings
Courier	Read only assigned orders and add status updates with encrypted notes.	Dashboard · Delivery queue · Order detail · Notifications
Public	Look up a tracking ID and verify status without seeing personal data.	Track · Benchmarks · Privacy model

7.2 Page map

Page	Role	What it does
`/login`	any	One-click Anvil personas or injected wallet; chain status; entry points to tracking and benchmarks.
`/dashboard`	all signed-in	Role-aware overview: active, in transit, delivered, exceptions, next delivery, activity, dispatch list.
`/orders`	customer, admin	Searchable/filterable order list; admin sees all, customer sees own; responsive cards.
`/orders/new`	customer, admin	Three-step encrypted order wizard with local review; admin selects the customer.
`/orders/:id`	authorised readers	Route, parcel, journey, timeline, key holders, chain record and role-specific actions.
`/queue`	courier	Only assigned orders, filtered by action state, with quick update.
`/dispatch`	admin	Unassigned orders, courier workload, key hand-off and assignment dialog.
`/team`	admin	Courier role management, key status, customers, admins.
`/reports`	admin	Volume, status mix, courier completion, top routes, benchmark entry point.
`/notifications`	all signed-in	Chain-derived activity feed with local read state.
`/settings`	all signed-in	Public display name, vault fingerprint, lock, theme, connection details.
`/track`, `/track/:id`	public	Status-only tracking by ID; deliberately limited.
`/privacy`	public	The envelope model, visibility table, derivation, honest limits.
`/benchmarks`	public	Caliper results, methodology, reproduction commands and native report link.
`/not-found`	any	Friendly 404 with tracking/privacy paths.

7.3 Product walkthrough

The screenshots below show the implemented interface.

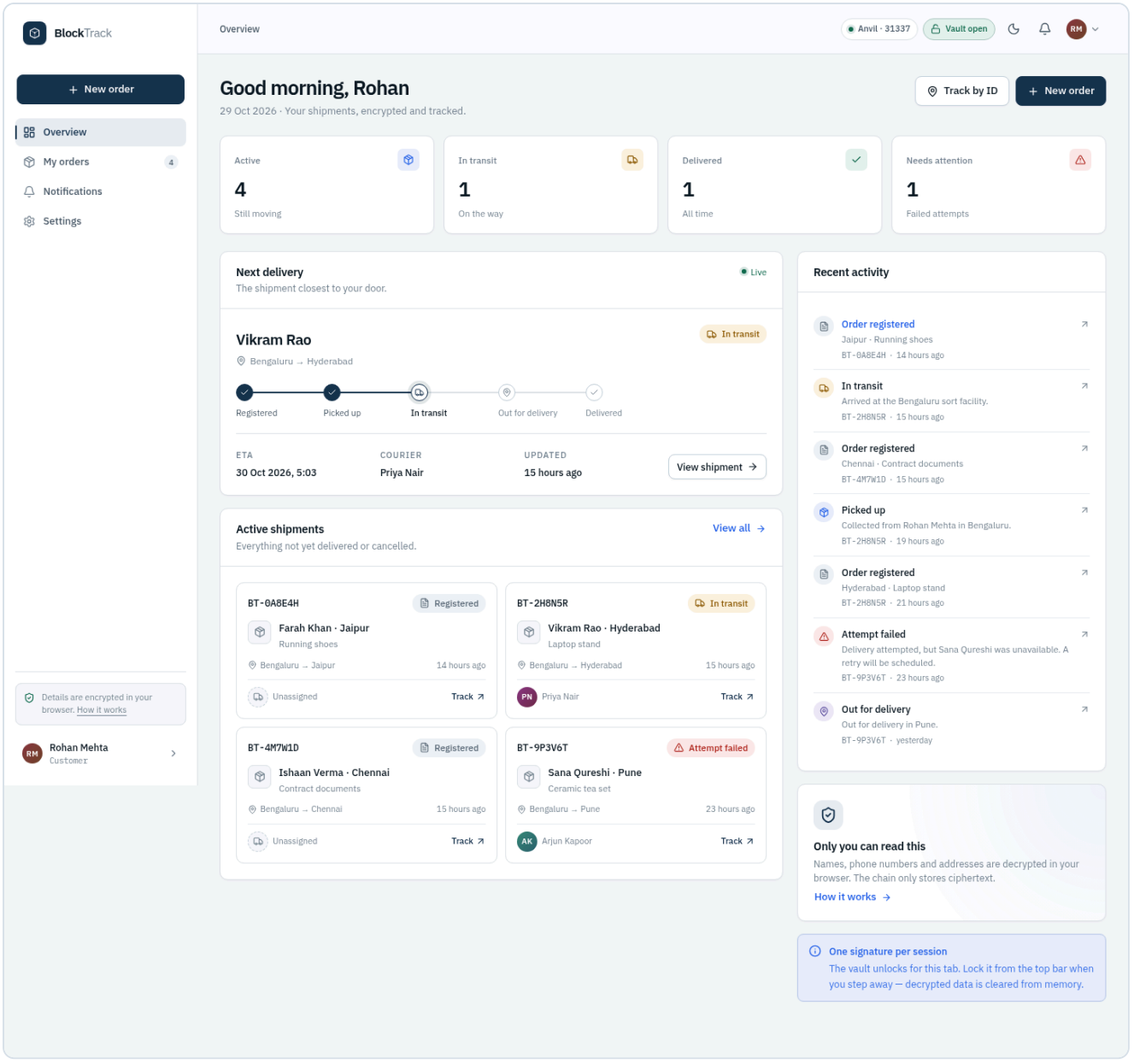


Figure 5 Customer dashboard. The next-delivery card combines status, ETA, courier and a progress rail; the activity feed is derived from decrypted chain events.

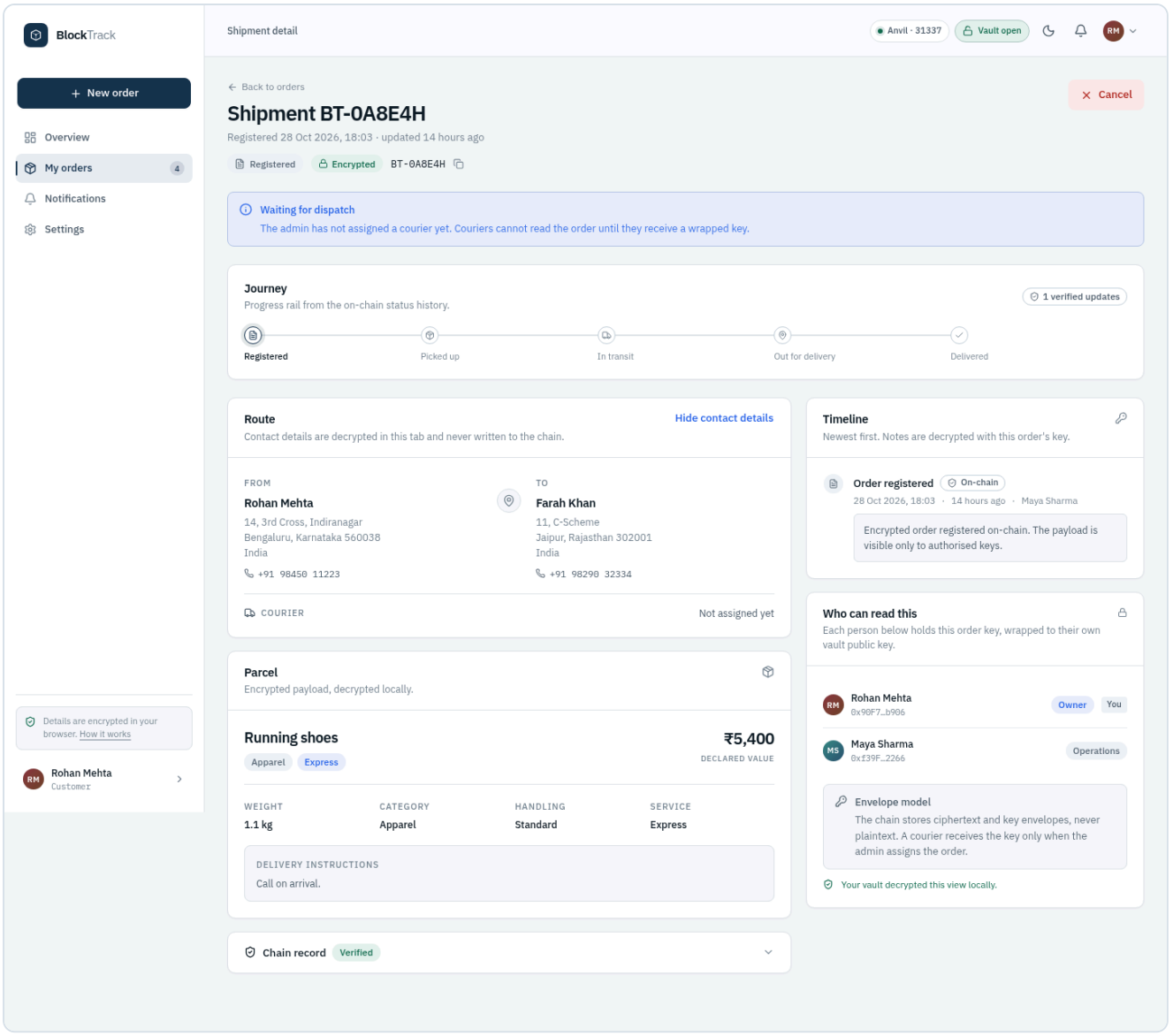


Figure 6 Order detail. The route and parcel are decrypted locally; the access panel lists every wallet with a key envelope; the chain record is expandable.

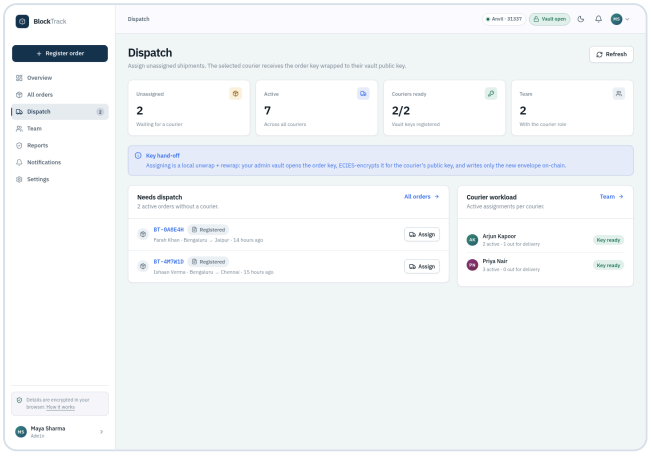


Figure 7 Dispatch. Assigning wraps the order key for the selected courier.

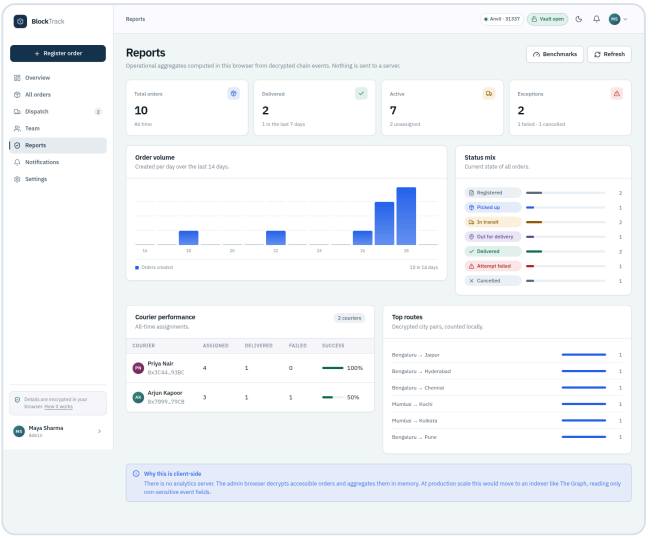


Figure 8 Reports. All aggregates are computed in the admin browser; no analytics server.

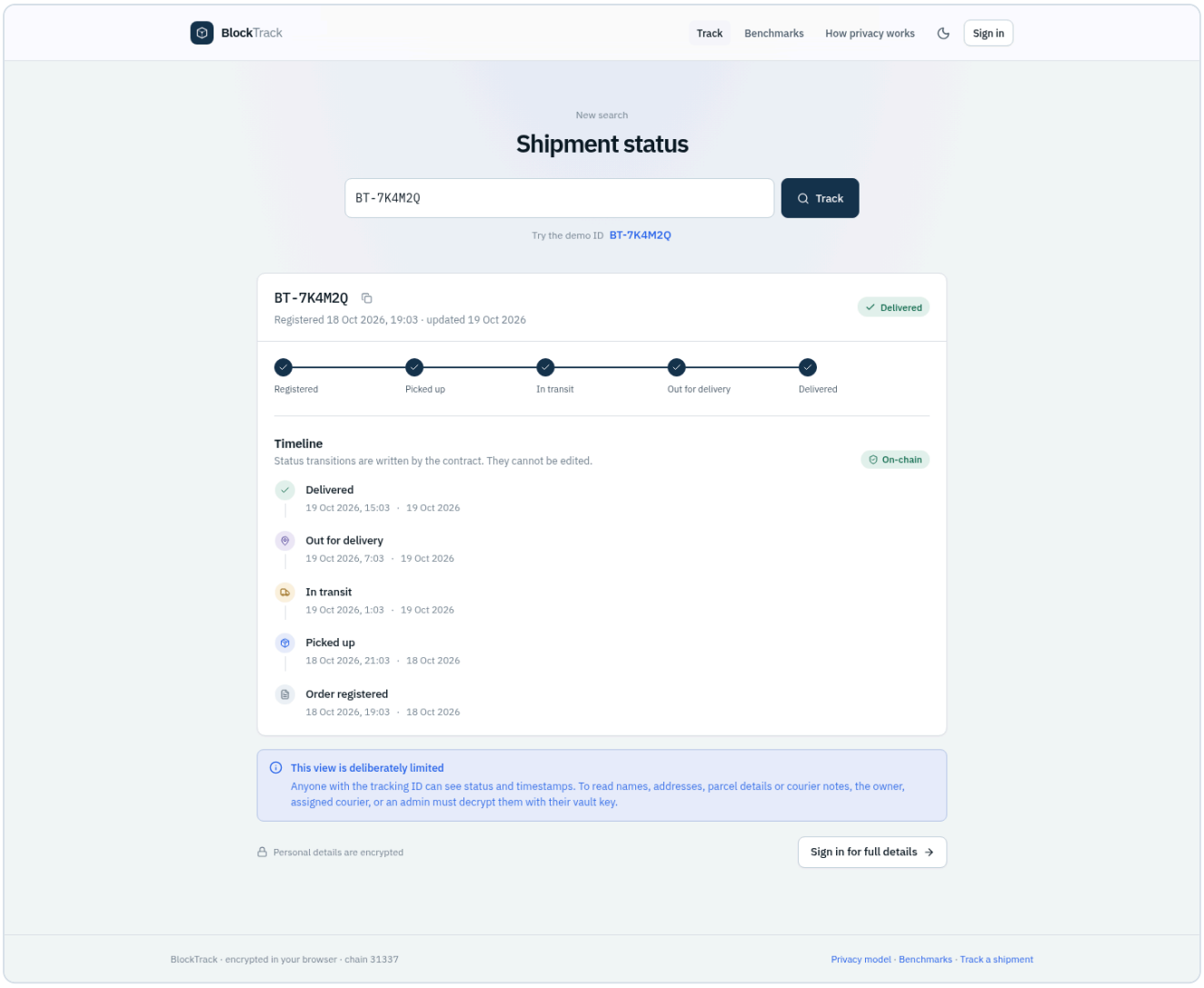


Figure 9 Public tracking. Status and timestamps are visible; names, addresses, parcel details and notes are absent.

7.4 The Cold Precision design system

The UI was rebuilt around a custom token system rather than a component library. The goal was a calm, operational interface for sensitive data: cool gray canvas, deep navy, one blue accent, IBM Plex Sans and Mono, and no decorative gradients or glass effects.

Consistency

One set of colours, spacing, radii, type sizes and focus rings. Status is always a tint + icon pair, not colour alone. Light and dark themes share the same structure.

Responsive

Desktop sidebar → off-canvas drawer → bottom navigation. Tables become cards, toolbars stack, and action buttons compact at mobile widths. The screenshots were captured at 1440 px and 390 px.

Accessible

Labelled fields, 44 px targets, visible focus, semantic headings, dialog titles, status text alongside colour, and reduced-motion support.

Privacy cues

The vault chip is always visible, contact details can be hidden in place, encrypted badges mark private values, and locking the vault clears decrypted views.

7.5 UX decisions worth calling out

- **One signature per session.** The first unlock signs a fixed message; local accounts unlock automatically; the top bar shows “Vault open” and locks in one click.
- **No plaintext preview before encryption.** The new-order review shows the data locally, then the button says “Encrypt & register”; the transaction only receives ciphertext and envelopes.
- **Role-appropriate actions.** A customer sees “Cancel” on a Created order; a courier sees “Update status”; an admin sees assign, update and cancel. The UI never offers an action the contract will reject.
- **Honest empty and error states.** No orders, no key, not found, wrong network, chain unreachable and wallet-rejected transactions all have specific copy.
- **Benchmarks as part of the product story.** The public benchmark page makes the evidence inspectable without a login and links to the native Caliper report.

08 Implementation and technology choices

The final stack is deliberately smaller than the synopsis. Fewer moving parts, fewer places for plaintext, and a test surface that covers the contract, the crypto and the browser.

8.1 Final stack

Layer	Choice	Why this choice
Chain	Anvil 31337 (local)	Deterministic, free, private and instant; a public testnet remains configurable.
Contract	Solidity 0.8.24, OpenZeppelin 5.7 AccessControlEnumerable	Audited role primitives; enumerable roles make the admin/team UI straightforward.
Contract toolchain	Foundry · Forge · Anvil	Solidity-native tests, fast feedback, no separate JS test harness for the contract.
Frontend	React 18.3 · Vite 5.4 · React Router 7 · TanStack Query 5	Small static SPA; server-state caching; no framework server.
Chain client	ethers 6.13	Typed contract access, event queries, browser signer support.
Crypto	WebCrypto AES-256-GCM + HKDF · eciesjs 0.5	Native authenticated encryption; ECIES over secp256k1 for key wrapping.
Wallet	Local accounts + injected wallet	One-click local accounts; a real wallet is supported for a public-testnet or production path.
UI	Custom CSS tokens · IBM Plex · Lucide	Consistent Cold Precision design without a component-library dependency.
Tests	Forge · node:test · Playwright-core + Chrome	Contract, pure crypto and real browser flows at the right layers.
Benchmarks	Hyperledger Caliper 0.6.0 + Ethereum adapter	Standard harness; isolated from runtime; produces a native report.
Version control	Git + GitHub · branch v2	Milestone history and a reviewable final state.

8.2 How the frontend talks to the chain

- **One contract wrapper.** `lib/contract.js` constructs ethers contracts from the synced ABI and the runtime `deployment.json`; `humanizeError` maps custom errors and wallet rejections to readable copy.
- **Event discovery.** `useOrders` queries `OrderCreated` and `CourierAssigned` logs, then loads the order, its wrapped key and its history. The tracking ID is non-indexed in the events so the client can read it; the owner/courier remain indexed for filtering.
- **Decrypt after fetch.** The order key is unwrapped with the vault key, the payload is decrypted with the tracking-ID AAD, and notes are decrypted per history entry. A corrupted or mismatched ciphertext becomes a recoverable error on the card, not a crashed page.
- **Query discipline.** Query keys include the contract, wallet and vault fingerprint. Polling refreshes role/order data; the cache is cleared on lock and on account/persona change so decrypted data does not survive a session boundary.
- **No server API.** Profiles are read from `getProfile`, roles from enumerable AccessControl functions, public tracking from status-only getters. The frontend derives notifications, reports and dashboards locally.

8.3 Vault and session lifecycle

At boot the app restores a local account or an injected wallet. A local account signs the fixed vault message locally and derives the key automatically; an injected wallet asks for one `personal_sign`. The public half of the derived key is compared with the on-chain profile. A mismatch produces a deliberate gate instead of a silent decryption failure. Locking the vault zeroises the key array, clears the query cache and unmounts decrypted views. Account and chain changes reset the session.

8.4 Implementation trade-off

The production bundle is approximately 924 KB (305 KB gzip) because ethers, eciesjs and every route are eagerly imported. Route-level code splitting is the obvious next optimisation; it was not allowed to delay the privacy work.

09 Key decisions and trade-offs

The design is best understood by being explicit about what was chosen, what was rejected and what each choice costs. None of the decisions below is free; each one buys a property the project needed.

Decision	Alternative rejected	Why chosen	Cost / mitigation
Public chain for audit	Centralized database	Tamper-evident shared history; no single operator can rewrite it.	Gas, latency and public metadata; mitigated by encryption and status-only views.
Encrypted payload on-chain	Plaintext parcel fields	Integrity without a public PII leak; the chain remains the only shared state.	8 KiB cap, gas and ciphertext lengths; production would anchor an IPFS/Arweave CID.
Per-order key + ECIES	Contract-level read ACL	Read access becomes cryptographic; no server key store; one key per order limits blast radius.	Revocation is not retroactive; recovery is hard; documented in the privacy and limitations sections.

Decision	Alternative rejected	Why chosen	Cost / mitigation
Signature-derived vault	Password + server KMS	No password, no server, no recovery database; the wallet is the identity.	One signature per session; wallets must be deterministic; no recovery if the wallet is lost.
No backend	API + database	No plaintext server, no second source of truth, static hosting.	Client-side event scanning, no server notifications/search; an indexer is future work.
Local chain demonstration	Public-testnet deployment	Deterministic, free, private, reproducible; no faucet dependency.	The submitted evidence is local; a public-testnet deployment is future work.
Foundry	Hardhat	Fast Solidity-native tests, Anvil, clean deploy/seed scripts.	Different JS tooling ecosystem; the frontend still has its own tests.
Public status-only	Full public history	Protects names, relationships and notes; the public can still verify status.	Less public transparency; actor identity is available only to authorised readers.
Admin key custody	No admin key / threshold	Operations needs to dispatch and support every order.	Admin is a trusted reader; production should use threshold or split roles.
Custom design system	Tailwind / component library	Consistent Cold Precision language; no generic dashboard look; full control of states.	More CSS to maintain; documented tokens make it coherent.
Optional Caliper harness	No benchmark	Standard tool, repeatable report, quantitative evidence for the contract path.	Deprecated Ethereum adapter; isolated from runtime.

The main engineering decisions. The four headline decisions are expanded below.

9.1 Why no backend?

The backend is where a normal tracking system would store plaintext, manage sessions and index events. It is also where the privacy claim would break. Removing it means a stolen server cannot leak shipment details, and there is no account database to secure. The browser becomes the only place a plaintext order exists. The cost is real: no server-side search, no push notifications, no password reset, and more work in the client. At prototype scale those costs are acceptable; at production scale an indexer that reads only public fields can be added without reintroducing plaintext.

9.2 Why encrypt the on-chain payload?

A blockchain is permanent by design, which makes it the worst place for a mistake with personal data. If the payload were plaintext, the audit trail would become an irreversible public leak. Encrypting the payload keeps the chain useful for ordering, status and key envelopes while making the content useless to an observer. The trade-off is that the contract cannot inspect or validate the plaintext, and the client must handle decryption failures. The AAD binding and authenticated encryption make tampering detectable.

9.3 Why a local chain for the demonstration?

The demonstration runs on a local Anvil chain so every test, screenshot and seeded order is reproducible without faucets, confirmation delays or a changing public state. The contract and frontend are network-agnostic; a public-testnet deployment is a configuration step and future work, not a contract change.

9.4 Why does the admin hold every key?

Dispatch and support are impossible if operations cannot read an order. The contract enforces that the primary admin is a key holder at creation, and the admin holds a wrapped key for every order. This is a deliberate trust boundary, not an oversight. A production system should split operations duties, use a threshold or multi-party key manager, and make admin access auditable. The prototype keeps one admin for clarity and operational reliability.

9.5 What the design buys

- ✓ Tamper-evident status history without public PII.
- ✓ Role-enforced writes and cryptographically scoped reads.
- ✓ A full user flow with no server, no passwords and no plaintext database.
- ✓ A reproducible local deployment with real tests, real encryption and real transactions.
- ✓ Quantitative contract evidence through Caliper.
- ✓ A coherent, accessible interface that feels like a product, not a prototype.

> THE DESIGN IN ONE SENTENCE
>
> BlockTrack does not try to be a production logistics platform; it proves a credible privacy architecture on a working Ethereum chain, measures the part that can be measured, and states the boundaries of the part that cannot.

10 Testing and quality assurance

The test strategy follows the risk surface: the contract is where permissions and state live, the crypto module is where confidentiality lives, and the browser is where the full flow either works or does not.

Layer	Result	Tooling	What it covers
Contract	`34/34`	Foundry / Forge	Roles, profiles, order creation, holders, key grants, assignment, state transitions, cancellation, public getters, limits, timestamps.
Cryptography	`7/7`	Node `node:test`	Vault determinism, public key, ECIES wrap/unwrap, payload round trip, AAD binding, tamper rejection, note AAD, order-key uniqueness.
Browser E2E	`21/21`	Playwright-core + Chrome	Create, assign, courier update, customer decrypt, public no-PII, admin cancel, vault lock/unlock.
Visual QA	`24 views`	Playwright screenshots	Desktop, dark and mobile; no browser errors; responsive and dark token checks.
Build	`Pass`	Vite production build	Static bundle compiles and serves from the generated output.
Benchmark	`2/2`	Hyperledger Caliper	50 order writes and 100 public reads with zero failures.

Verification evidence. The end-to-end flow is the strongest single piece of evidence; the Forge suite is the broadest contract check.

10.1 What the contract tests prove

Authorisation and roles

- Only the admin can add/remove couriers and create for a customer.
- Only the assigned courier or an admin can update.
- Only the owner (before pickup) or admin can cancel.
- A revoked courier cannot continue updating.

State and data

- Every legal transition, and rejection of illegal ones.
- Final orders cannot move; failed attempts can retry.
- Owner/admin holders are required; duplicates and zero addresses rejected.
- Public getters expose status/timestamps, not ciphertext or notes.

10.2 What the Browser E2E proves

1. A customer can create an encrypted order and immediately read it back.
2. The admin can see the decrypted order and assign a different courier.
3. The courier can see the assignment in their queue and update it with an encrypted note.
4. The customer can read the courier note and courier identity; the public tracker cannot.
5. An admin can cancel a non-final order; the reason is decrypted for authorised readers.
6. Locking the vault removes order data from the screen; signing again restores it.

10.3 Verification boundaries

- No automated real-MetaMask test; the automated flow uses the local Anvil accounts, while the injected-wallet path is implemented.
- No public-testnet end-to-end test; the submitted evidence is local.
- No fuzzing or formal verification; the contract tests are scenario-based unit tests.
- No production load test. Caliper is a single-worker microbenchmark, not a saturation test.
- No external crypto audit. The design is careful; the implementation is not certified.

11 Benchmarking with Caliper

Caliper was added as an optional, isolated harness. It measures the contract and RPC path; it deliberately does not claim to measure browser encryption or UI work.

11.1 The run

Round	Tx	Success	Send rate	Avg latency	Throughput
Create order 200-byte dummy ciphertext, two 129-byte key envelopes	50	50 / 0	5.1 TPS	0.04 s	5.1 TPS
Public status read status-only getter on a pre-created order	100	100 / 0	20.2 TPS	0.00 s	20.2 TPS

Caliper 0.6.0 on local Anvil, one worker, fixed-rate workload; 2/2 rounds passed in 32.1 s.

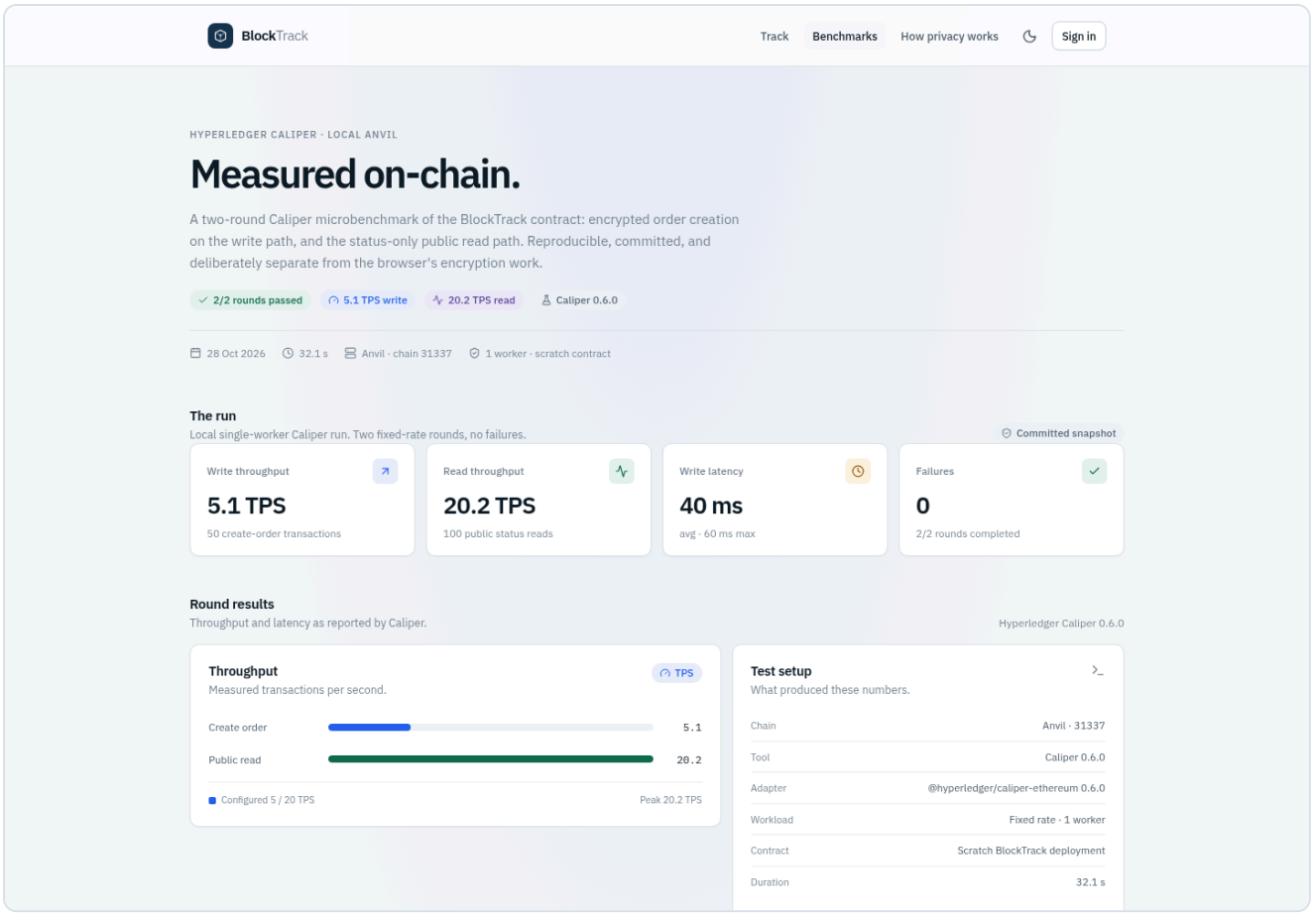


Figure 10 The public benchmark page. It presents the same run with the methodology and a link to the native report.

11.2 How to read the numbers

The workload is fixed-rate, so the measured throughput is expected to match the configured rate. The useful signals are the zero failure count, the low write latency on Anvil and the order-of-magnitude difference between a write and a read. This is a sanity check and a demonstration of the toolchain, not a capacity claim. A stronger benchmark would sweep rates, run multiple workers, and separate contract execution from RPC and confirmation latency.

11.3 Benchmark boundary

- **Chain-level scope.** Caliper sees the contract and RPC, not AES-GCM, ECIES, wallet signatures or React.
- **Dummy ciphertext.** The contract validates lengths, holders and roles; the write round exercises that path without pretending to encrypt a real order.
- **Deprecated adapter.** `@hyperledger/caliper-ethereum` 0.6.0 is the latest published version but is marked deprecated and uses web3.js 1.2. It is suitable for a local demonstration, not a production benchmark.
- **Runtime independence.** The application bundle does not import Caliper. The harness deploys a scratch contract and does not touch the application data.

12 Deployment and limitations

The submitted build runs on a local Anvil chain with deterministic, fictional data. This makes the demonstration reproducible and keeps the privacy claim honest: no real personal data is published to a public network.

12.1 Delivered environment

Component	Delivered state
Chain	Anvil, host `127.0.0.1`, port `8545`, chain ID `31337`, with a backdated genesis for a realistic timeline.
Contract	A fresh deployment on startup; the address is supplied to the frontend through a runtime configuration file.
Data	10 seeded orders, 7 assignments, 21 updates and six fictional local accounts.
Frontend	Static React application; no backend, database, API or account server.
Public network	Not part of the submitted build. The contract and frontend are network-agnostic and can be configured for a public testnet.

12.2 Limitations

Limitation	Why it matters	Future work
Local-chain evidence	The submitted evidence is from a local Anvil network, not a public testnet.	Deploy to a public testnet with a funded wallet and a public RPC; run the same end-to-end flow.
Admin key custody	Operations can read every order by design; this is a wide trust boundary.	Threshold or multi-party key management; split support and dispatch roles.
Public metadata	Addresses, timing, order frequency and ciphertext size can be correlated.	Relaying, address rotation, padding and a privacy review before real deployment.
Revocation	Reassignment removes the on-chain envelope but cannot recall plaintext already read.	Order-key rotation, re-encryption and new envelopes for the remaining readers.
No recovery or rotation	Losing the wallet loses the vault key; a compromised key exposes everything wrapped for it.	Deliberate social recovery and versioned, rotatable vault keys.
Unaudited	The contract and client cryptography have not been externally audited.	External audit, formal threat-model review, fuzzing and invariant testing.
Client-side scale	Event scanning and aggregates run in the browser and will not scale to large order volumes.	An indexer that reads only non-sensitive fields; decryption stays local.
No real logistics integration	The project does not connect to couriers, GPS, payments or a production operations system.	A production integration layer and a full security/compliance review.

12.3 Future work

- Public-testnet deployment with a real wallet and a status-only public sample order.
- External security review of the contract and the key-envelope scheme.
- Threshold admin keys and a documented rotation/revocation procedure.
- Off-chain encrypted storage with an on-chain CID, and an indexer for discovery.
- Production logistics integration, performance work and a privacy compliance review.

13 Team contributions

The project was delivered by a nine-member team. The table records the final responsibility area and the main contribution of each member.

Member	Role	Contribution
Shantanu Bhargava	**Smart Contract & Protocol**	Contract data model, role-based access, key-envelope protocol, status machine and the 34-test contract suite.
Ankur Kumar	**Frontend & Design System**	Cold Precision tokens, application shell, UI primitives, page composition, responsive and dark themes.
Rijul Kaushik	**Blockchain Integration**	Wallet and vault providers, ethers contract layer, event indexing, transaction flows and error handling.
Ananya Sahani	**Client Data & Privacy**	Client cryptography integration, vault lifecycle, query caching, local notifications and the privacy data model.
Hemesh Kukreja	**Admin Operations**	Dispatch, courier workload, assignment dialogs, team role management, reports and settings.
Sandeep Balaji	**Tracking & Customer Experience**	Public tracker, customer order detail, timeline, progress rail and tracking search.
Adhisha Sahoo	**QA & Test Automation**	Test plan, crypto and browser automation, visual QA, accessibility checks and benchmark validation.
Kshitij Luthra	**Deployment & DevOps**	Local Anvil stack, deployment and seeding, build configuration and the Caliper harness.
Vansh Pahwa	**Integration & Documentation**	Module integration, README, privacy model, benchmark page, final demonstration and this report.

All members participated in integration, review and the final review.

14 Conclusion

BlockTrack started as a simple “put parcel tracking on a blockchain” idea. The implementation made the problem more interesting: a public audit trail is only useful if the data underneath it can stay private.

The final system answers that with a clear architecture. The contract owns the shared facts — owner, courier, status, timestamps and key envelopes. The browser owns the keys and the plaintext. The customer, operations and the assigned courier can read an order; the public can verify its status; an unassigned wallet cannot. The UI is a complete, role-aware product rather than a single admin table, and the evidence is reproducible: contract tests, crypto tests, a browser E2E flow, visual QA and a Caliper benchmark.

The project also records what it does not solve. A public-testnet deployment is future work. The admin is a trusted reader. Metadata remains public. Revocation is not retroactive. The contract and crypto are unaudited. These are not footnotes; they are the boundary of the claim. A production system would need key management, off-chain encrypted storage, an indexer, audits and a legal review.

FINAL STATEMENT

BlockTrack demonstrates that privacy can be an architectural property of a blockchain DApp, not just a policy — and that a prototype can be honest about the difference between what it proves and what it does not.

15 References

[1] Ethereum Foundation. *Ethereum Developer Documentation.* https://ethereum.org/developers/

[2] Solidity Team. *Solidity Documentation.* https://docs.soliditylang.org/

[3] OpenZeppelin. *Access Control.* https://docs.openzeppelin.com/contracts/5.x/access-control

[4] Foundry. *Foundry Book — Forge, Cast, Anvil.* https://book.getfoundry.sh/

[5] Foundry. *Anvil — Local Ethereum Node.* https://book.getfoundry.sh/anvil/

[6] Meta. *React Documentation.* https://react.dev/

[7] Vite. *Vite Documentation.* https://vite.dev/guide/

[8] Remix Software. *React Router Documentation.* https://reactrouter.com/

[9] Tanner Linsley. *TanStack Query Documentation.* https://tanstack.com/query/latest

[10] ethers.js. *ethers v6 Documentation.* https://docs.ethers.org/v6/

[11] eciesjs. *ECIES for JavaScript.* https://github.com/ecies/js

[12] Hyperledger. *Caliper Documentation.* https://hyperledger.github.io/caliper/

[13] npm. *@hyperledger/caliper-ethereum.* https://www.npmjs.com/package/@hyperledger/caliper-ethereum

[14] Ethereum Improvement Proposals. *EIP-170: Contract code size limit.* https://eips.ethereum.org/EIPS/eip-170

[15] Ethereum Improvement Proposals. *EIP-191: Signed Data Standard.* https://eips.ethereum.org/EIPS/eip-191

[16] H. Krawczyk and P. Eronen. *RFC 5869: HKDF.* https://www.rfc-editor.org/rfc/rfc5869

[17] NIST. *SP 800-38D: Galois/Counter Mode (GCM).* https://csrc.nist.gov/pubs/sp/800/38/d/final

[18] The Graph. *Indexing Ethereum data.* https://thegraph.com/docs/